InvoraLite — Product Quality Review

Review date: 12 July 2026 | **Exported:** 12|07|2026 | **Version:** 1.0.0 | **Stack:** Tauri 2 + React 19 + SQLite

Executive Summary

Overall Rating: 8.5 / 10

InvoraLite is a mature v1.0 offline desktop inventory application for a single store, with unusually broad coverage for a "lite" product: sales, purchases, Rate Master multi-level UOM pricing, credit, supplier advances, GST invoices, analytics, accounting helpers, tax reports, and a complete print ecosystem.

Since the prior review, the product gained a full **Rate Master** module (2–4 unit levels, dated rate history, locked expired periods), deep **Purchase ↔ Rate Master** linking (auto cost/selling rates), **Sales UOM auto-conversion** (e.g. 30 Piece → 1 Tray), and **People payment history** tracking. Earlier releases already included database encryption at rest, encrypted backups, and frontend role-based UI gating.

The app is **ready for trusted single-store deployment** on Windows. Remaining gaps are mainly operational and scale-related: license secrets in source, JSON-blob data model, Manager/Rate Master RBAC drift between UI and Rust, thin UI/E2E test coverage, and no code signing or auto-update.

Rating Breakdown

Area	Score	Notes
Feature breadth	**8.8/10**	Sales, purchases, Rate Master, stock, returns, people + payment history, invoices, analytics, accounting, tax, office, backup, licensing
UX / polish	**8.0/10**	Cohesive Invora theme; aligned Rate Master / Purchase forms; payment history modal; role banners; some large page files remain
Business logic	**8.3/10**	UOM hierarchy, dated rates, purchase→stock→RM cost sync, sale pack auto-convert, supplier AP, customer credit FIFO
Security	**7.8/10**	Argon2, vault, backups, audit, lockout; UI RBAC strong; Rust allowlist gaps for Manager / rate masters
Data / reliability	**6.5/10**	Health checks, encrypted vault, backup/restore; JSON-in-SQLite limits integrity at scale
Testing & quality	**6.0/10**	Solid domain unit tests (Rate Master, UOM, GST, payments); sparse page/E2E coverage; no CI
Deployment	**7.5/10**	Windows NSIS installer with dated exports; user manual + product review PDFs; no code signing or auto-update

Technology Stack

Layer	Technology
Desktop shell	Tauri 2, WebView2
Frontend	React 19, TypeScript 5.6 (strict), Vite 6, Tailwind 4
Charts	Recharts 2.15
Backend	Rust 2021 — src-tauri/src/
Database	SQLite via rusqlite (bundled, WAL); domain data as JSON blobs in app_storage
Encryption	AES-256-GCM (vault), Argon2id (backup passwords), Windows DPAPI (key file)
IPC	Tauri commands (storage, audit, auth, license, backup, password)

Architecture

```
React (pages, components, lib/data.ts, rateMaster.ts, usePermissions)
  → storage.ts (username + role as StorageContext)
    → Tauri commands
      → DatabaseManager
        → validation · sql_guard · rbac · storage_repository · audit
        → encryption_key (seal/unseal vault on shutdown)
        → backup_archive (v1 plaintext + v2 encrypted)
```

Bootstrap: Splash → License gate → Setup → Login → Main layout.

Feature Completeness

Implemented (substantial)

Module	Highlights
License	60-day trial, device-bound keys, ZIP activation
Auth / setup	Business profile, GST, staff accounts, session timeout
Dashboard	Stats, charts, low-stock alerts, quick New Sale (role-gated)
Products	CRUD, categories, IMEI, low-stock threshold, CSV, stock adjustments
Rate Master NEW	2–4 UOM levels; Unit 1 cost + selling prices; dated history; locked expired periods; blank sell = not sold at that rate code
Purchases	Multi-line GRN; Rate Master link; locked RM cost; separate SKU / Barcode / Brand; shipping expense sync
Sales	Cash / e-pay / credit / partial; RM UOM pricing; auto-convert qty to matching pack; half-unit support; thermal receipt
People	Customers & suppliers; credit; Record Payment; **Payment History** (date, amount, mode, balance after)
Invoices	Print, credit settlement, void with stock restore (password confirm)
Analytics	Weekly / monthly / yearly earnings
Accounting	Chart of accounts, journal helpers, P&L, monthly close, tax submission report
Office	Expenses, assets, expense reports
Settings	Staff roles, business profile, audit log, encrypted backup
Documentation	User manual PDF, product review PDF, dated installer exports

Rate Master ↔ Purchase ↔ Sales (recent)

- **Rate Master:** hierarchy levels fixed after create; history shows carton cost/price and sell rates; Unit 1 cost editable on RM.
- **Purchase:** selecting an RM item fills UOM, selling rates (read-only strip), and locks cost when RM Unit 1 cost exists; save writes purchase cost back to RM when entered.
- **Sales:** UOM list from priced RM units; entering a qty that equals a pack size auto-upgrades (30 Piece → 1 Tray, 7 Tray → 1 Carton).

Remaining gaps

- Multi-store / cloud sync — not supported (single-computer design)
- Code signing and auto-update — not configured
- License secrets in source — need rotation before wide distribution
- E2E and CI pipeline — not present
- macOS / Linux bundles — Windows NSIS only
- Relational schema — core entities still JSON blobs in app_storage

- RBAC drift: UI includes Manager + Rate Master for Store Keeper; Rust allowlists may not fully match — verify before production rollout

Role enforcement

Role	Products / RM	Purchases	Sales	Customers	Suppliers	Delete	Admin settings
Admin	✓	✓	✓	✓	✓	✓	✓
Manager	✓	✓	✓	✓	✓	✓	Staff / office / audit
Store Keeper	✓	✓	✓	✓	✓	✗	Business view only
Cashier	View	✗	✓	✓	✗	✗	Business view only
Viewer	View	✗	View	View	View	✗	Business view only

Security Posture

Strengths

Control	Implementation
Centralized DB access	DatabaseManager — UI never touches SQLite directly
Database encryption at rest	AES-256-GCM vault; DPAPI-protected key; sealed on exit
Encrypted backups	v2 Argon2id + AES portable exports; machine-key auto backups
RBAC	Role-based UI gating; password confirm for destructive actions
Password hashing	Argon2id
Account lockout	Failed-login lockout policy
Audit trail	Viewable in Manage → Audit Log
Session timeout	Idle logout

Remaining risks

Risk	Severity	Detail
License secrets in source	High	Rotate before wide distribution
UI vs Rust RBAC drift	Medium–High	Manager / Rate Master writes may fail on desktop while UI allows them
Browser / localStorage path	Medium	Dev/browser fallback lacks vault/RBAC
No code signing	Medium	Installer may trigger SmartScreen
Runtime plaintext DB	Low–Medium	Decrypted while app is running

Testing Coverage

Frontend (Vitest): Domain libs — Rate Master (levels, history, sale-unit auto-convert), inventory UOM, GST, payment balances, permissions, password policy, reports.

Rust: RBAC, storage/integrity, encryption, backup archive tests.

Gaps: Limited page/component tests for Purchase, People, Rate Master UI, and New Sale modal; no E2E; no CI pipeline.

Print & Reporting

Document	Format
Sale receipt	80mm thermal — preview then print
Tax invoice	A4 — Invoice page
Customer payment receipt	A5 landscape — FIFO + balance
Tax submission report	A4 — P&L, GST, registers
User manual	PDF — docs/
Product review	PDF — docs/ + exports/

Build & Deployment

Item	Detail
Dev	npm run tauri:dev
Build	npm run tauri:build → NSIS installer
Docs export	npm run review:pdf / npm run docs:pdf
Targets	Windows NSIS (x64)

Recommended Improvements

Before wider distribution

1. Align Rust RBAC with UI roles (Manager + Rate Master keys)
2. Replace license secrets; remove hardcoded ZIP password
3. Code-sign the Windows installer
4. CI running npm test + cargo test on every release

Medium term

5. Relational schema migration for core entities
6. E2E smoke tests for sale, purchase, Rate Master, backup, and role flows
7. Auto-update channel for patch releases

Changes Since Last Review (04|07|2026 → 12|07|2026)

Item	Prior status	Current status
Rate Master	Absent / early	Done — 2–4 levels, history, lock, cost + sell rates
Purchase ↔ RM	Basic purchase	Done — linked rates, locked cost, SKU/Barcode/Brand row
Sales UOM	Manual unit	Done — RM units + qty auto-convert to pack
Payment history	Totals only	Done — dated payment ledger modal
Overall rating	8.4 / 10	**8.5 / 10**

Use-Case Fit

Scenario	Fit
Owner's shop, trusted staff, multi-UOM goods (e.g. eggs)	**9/10** — Rate Master + purchase/sales flow is a strong daily tool
Few clients, supervised rollout	**8/10** — viable with license rotation + backup discipline
Many businesses / sensitive data	**6.5–7/10** — encryption helps; needs license hardening, RBAC align, CI
Multi-store / enterprise	**4/10** — JSON model and single-PC design not ready

Top Strengths

1. Coherent Rate Master → Purchase → Sales UOM pricing for pack-based inventory
2. Real shop workflows — credit, partial pay, supplier advances, Bhutan e-pay, GST, IMEI
3. Security foundation — encrypted DB, encrypted backups, password confirms, audit
4. Offline-first desktop with dated Windows installer and documentation PDFs
5. Complete print ecosystem — invoices, thermal receipts, payment receipts, tax reports

Top 3 Next Steps

1. **Align Rust ↔ UI RBAC** — Manager role and Rate Master storage writes
2. **Production license hardening** — rotate secrets, externalize signing key
3. **CI + E2E smoke** — sale, purchase, Rate Master, backup, role restriction

---

InvoraLite — Product Quality Review · Version 1.0.0 · EDP IT Department
See also docs/DATABASE_ARCHITECTURE.md and docs/InvoraLite_User_Manual.pdf

12|07|2026